Classification supervisée

Fiche TP3

W. Heyse

Contents

1. Installer le package `mvtnorm`: ce package permet de simuler des lois normales multivariées et de calculer les densités de loi normale multivariée.
2. Réaliser 1000 simulations de $Y \sim \mathcal{B}(1; 0,5)$ en vous aidant de la fonction `rbinom`.
3. Simuler $X = \begin{pmatrix} X_1 \\ X2 \end{pmatrix}$ sachant Y (utiliser la fonction `rmvnorm`) :
   - $X/Y = 0 \sim \mathcal{N}\left(\begin{pmatrix} 0 \\ 0 \end{pmatrix}, \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}\right)$
   - $X/Y = 1 \sim \mathcal{N}\left(\begin{pmatrix} 2 \\ 2 \end{pmatrix}, \begin{pmatrix} 1 & 0.8 \\ 0.8 & 1 \end{pmatrix}\right)$
4. Représenter grapiquement les données simulées.
5. Calculer pour l'individu de coordonnées $x = (1, 2)$ les $f_i(x)$ (densité de probabilité dans chacune des classes) (on pourra utiliser la fonction `dmvnorm`).
6. En déduire les probabilités d'appartenance de l'individu $x = (1, 2)$ aux classes.
7. A l'aide de la règle du maximum a posteriori, donner la classe prédite de cet individu.
8. Faire le calcul des probabilités d'appartenance pour l'ensemble des individus de la matrice de données X (la fonction `dmvnorm` peut prendre une matrice pour argument). Affecter les individus aux classes.
9. Représenter vos données en fonction des classes prédites et comparer avec les vraies classes. On pourra utiliser le code (où `X` représente les données, `Y` représente les étiquettes et `Yp` les étiquettes prédites):

```
data_ggplot <- data.frame(X, Y = as.factor(Y), YPred = as.factor(Yp))
data_ggplot$Classement <- factor( 2*as.numeric(Y) + Yp, levels = c(0,1,2,3),
                                  labels = c("Y = 0, Ypred = 0", "Y = 0, Ypred = 1", "
                                             Y = 1, Ypred = 0", "Y = 1, Ypred = 1") )
ggplot(data = data_ggplot,
       aes(x = X1, y = X2, color = Classement, shape = Classement)) +
    geom_point(size = 2, alpha = 0.8) +
    labs(title = "Données simulées - Erreurs de classement (vert et bleu)") +
    theme_minimal() +
    scale_colour_brewer(palette = "Set1")
```

10. Calculer la matrice de confusion et le taux de mauvais classement.
11. A l'aide la la matrice de confusion, calculer la spécificité et la sensibilité de votre règle de classement.
12. Installer et charger le package `ROCR` puis, grâce aux probabilités d'appartenance calculées à la question 6, tracer la courbe ROC de votre règle de classement et calculer sont AUC avec le code suivant :

```
pred_ROC = prediction(t_all[,2] ,Y)
perf_ROC <- performance(pred_ROC,"tpr","fpr")

plot(perf_ROC)

#Calcul de l'AUC
AUC_ROC <- performance(pred_ROC,"auc")

#Avec ggplot
data_ggplot <- data.frame(X = perf_ROC@x.values[[1]], Y = perf_ROC@y.values[[1]])

ggplot(data = data_ggplot, aes(x = X, y = Y)) +
      geom_line(size = 1) +
      labs(title = "Courbe ROC",
           subtitle = paste0( "AUC : ", round(AUC_ROC@y.values[[1]],4) )) +
      ylab(perf_ROC@y.name) + xlab(perf_ROC@x.name) +
      theme_minimal() +
      coord_fixed()
```

où$ `t_all`$ est la matrice qui contient, en colonne, les probabilités d'appartenance des individus aux classes 0 et 1.

13. Déterminer le seuil optimal de votre règle de classement avec la commande :

```
CutPoint = performance(pred_ROC, "cost", cost.fp = 1, cost.fn = 1)
SeuilOptimal <- pred_ROC@cutoffs[[1]][which.min(CutPoint@y.values[[1]])]
SeuilOptimal
```

On notera les arguments `cost.fp` et `cost.fn` qui permettent de changer les coûts de la règle de classement et qui ont donc une influence sur celle-ci. Ici, le coût de mauvais classement vaut 1, quelle que soit la classe.

On pourra aussi ajouter le seuil optimal au graphe précédent :

```
ggplot(data = data_ggplot, aes(x = X, y = Y)) +
      geom_line(size = 1) +
      geom_point(aes(x=perf_ROC@x.values[[1]][which.min(CutPoint@y.values[[1]])],
                     y=perf_ROC@y.values[[1]][which.min(CutPoint@y.values[[1]])]),
                colour="red", size = 2) +
      labs(title = "Courbe ROC",
           subtitle = paste0( "AUC : ", round(AUC_ROC@y.values[[1]],4) )) +
      ylab(perf_ROC@y.name) + xlab(perf_ROC@x.name) +
      annotate(geom="text",
               x=1.1*perf_ROC@x.values[[1]][which.min(CutPoint@y.values[[1]])],
               y=0.99*perf_ROC@y.values[[1]][which.min(CutPoint@y.values[[1]])],
               label="Couple (Se, 1-Sp) associé au seuil optimal",
               color="black", hjust=0) +
      theme_minimal() +
      coord_fixed()
```

14. Déterminer les classes d'appartenances de vos individus avec le seuil optimal, calculer la matrice de confusion et le taux de mauvais classement.